

Kuggle 세미나 3주차

분류 (Classification)

13기 이지유

Timeline



Chapter 1

분류란 무엇인가

분류의 개념, 활용 사례, 회귀와의 차이

분류 (Classification)

: 입력 데이터를 정해진 클래스(범주) 중 하나로
예측하는 지도학습의 한 종류
⇒ " 어느 그룹에 속하는가? "

분류가 필요한 이유

범주형 자료

숫자가 아닌 선택지(범주) 형태로
떨어지는 세상의 수많은 질문에 대응하기
위한 필수 도구

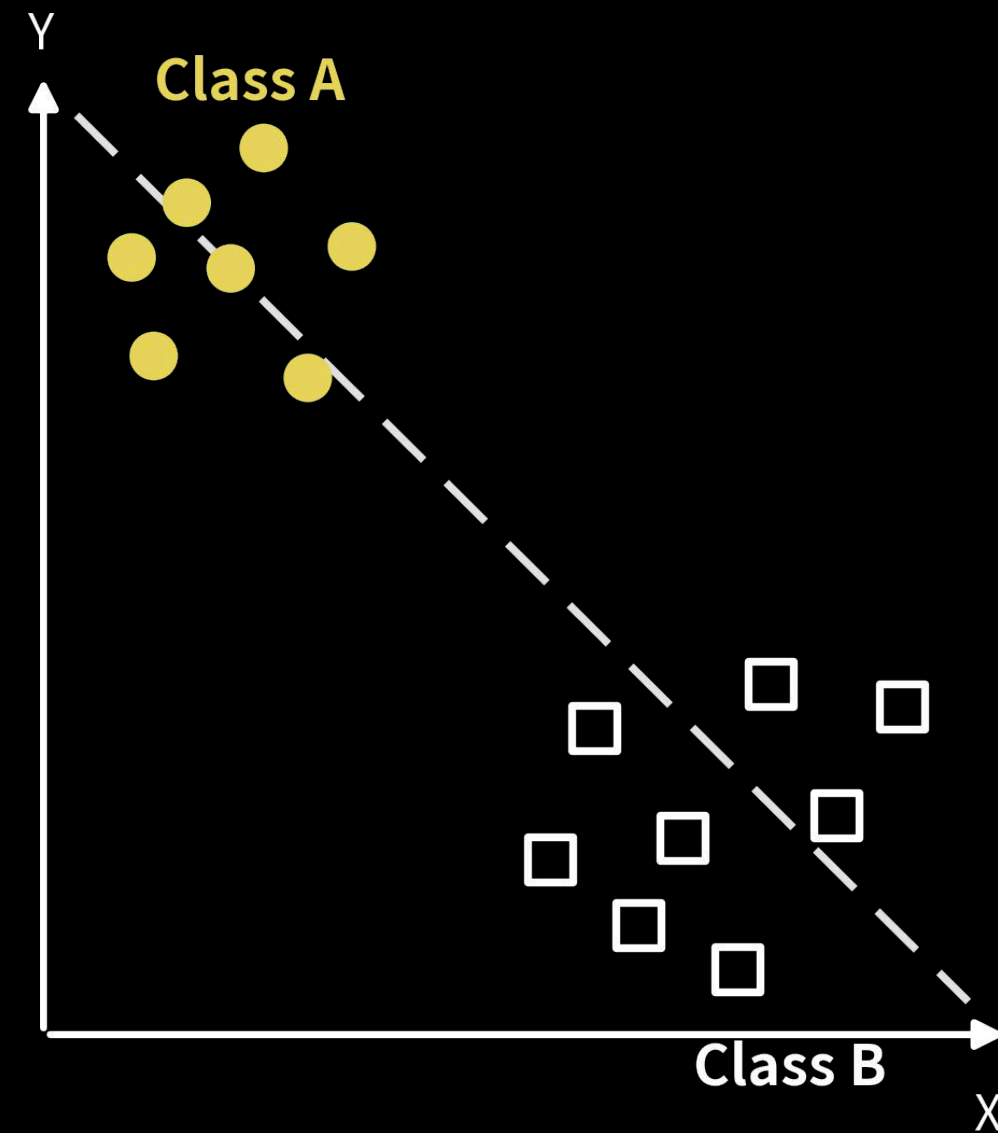
대규모 데이터

수백만 건에 달하는 방대한 데이터와
실시간 판단을 사람 대신 자동화로 처리

과거의 정답

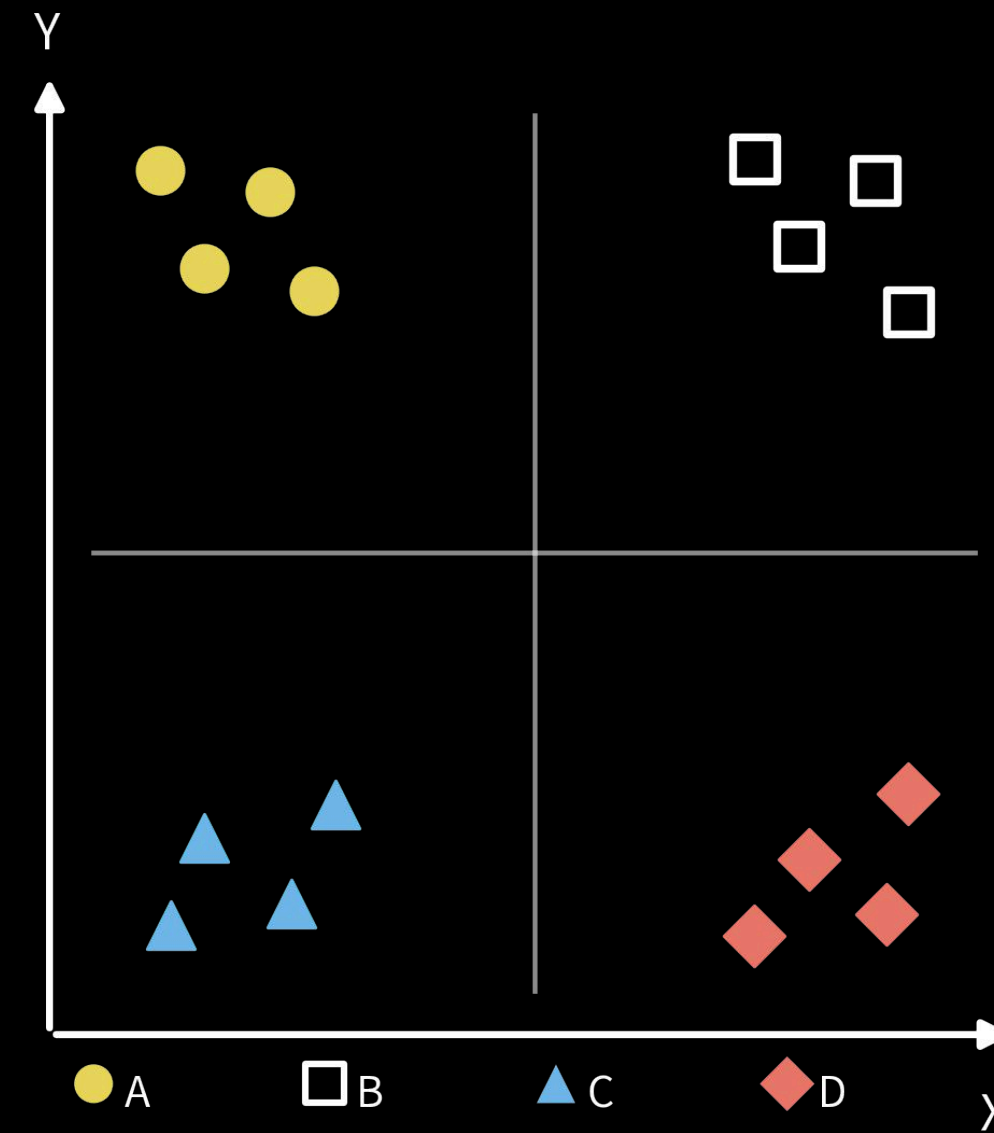
라벨(정답)이 포함된 과거 데이터가
축적되어 있어, 지도학습 기반의 분류를
적용하기 위한 최적의 환경

분류의 유형



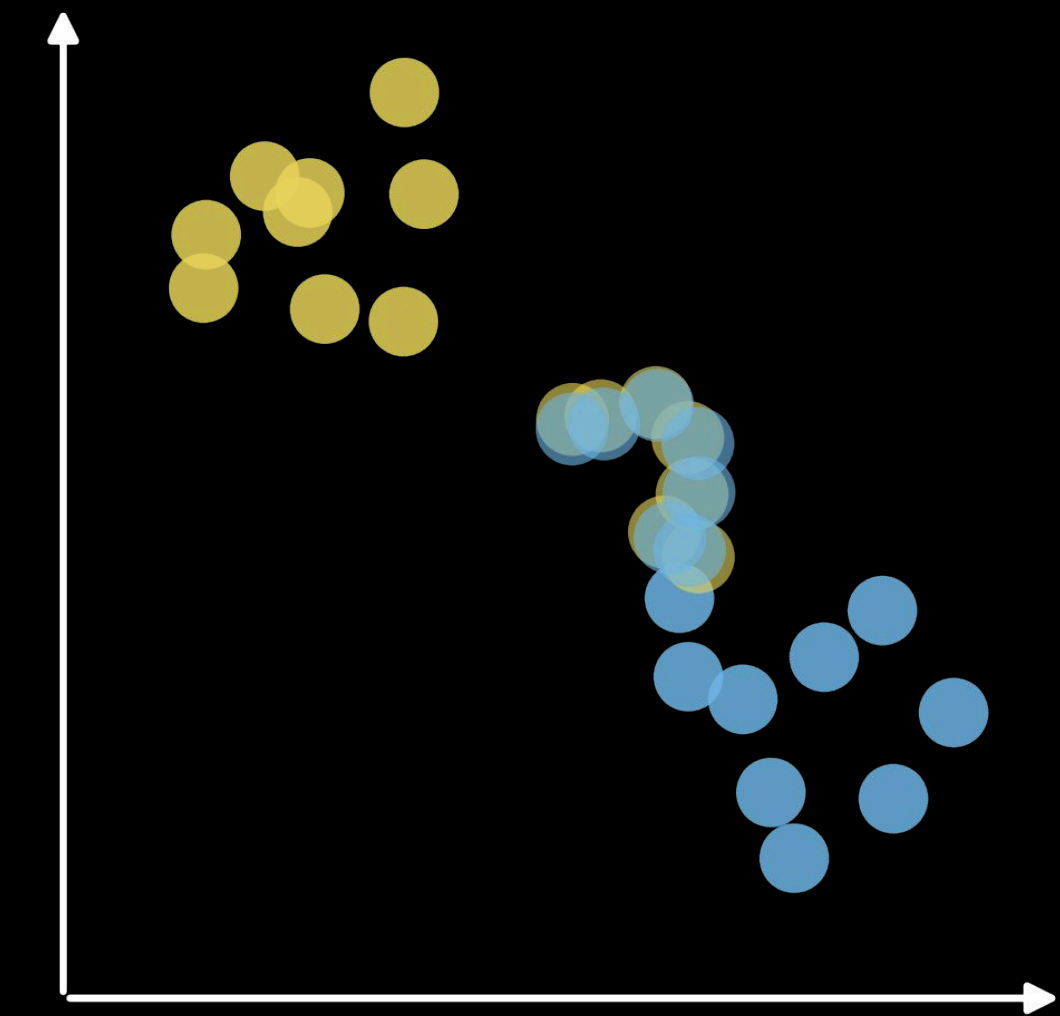
이진 분류 (Binary)

클래스가 2개인 경우
ex) 스팸 메일 판별, 암 진단(양성/음성), 이탈 예측



다중 분류 (Multi-class)

클래스 3개 이상, 샘플은 1개 클래스만 속하는 경우
ex) 붓꽃 품종 분류, 손글씨 숫자(0~9) 인식



다중 레이블 분류 (Multi-label)

한 샘플이 여러 클래스에 동시에 속할 수 있는 경우
ex) 뉴스 기사 분류 (정치+경제), 영화 장르 분류

분류 vs 회귀

분류 (Classification)

- 출력값: 불연속적인 값
ex) 스팸/정상, 합격/불합격, 붓꽃 품종
- 평가지표: Accuracy, Precision, Recall, F1, AUC
- 대표 모델: 로지스틱회귀, KNN, 트리, SVM

“어느 그룹인가”를 묻는다.

회귀 (Regression)

- 출력값 : 연속적인 숫자
ex) 내일 기온, 집값, 매출액 예측
- 평가지표: MSE, RMSE, MAE, R^2
- 대표 모델: 선형회귀, 회귀트리

“얼마나”를 묻는다.

Chapter 2

분류 알고리즘

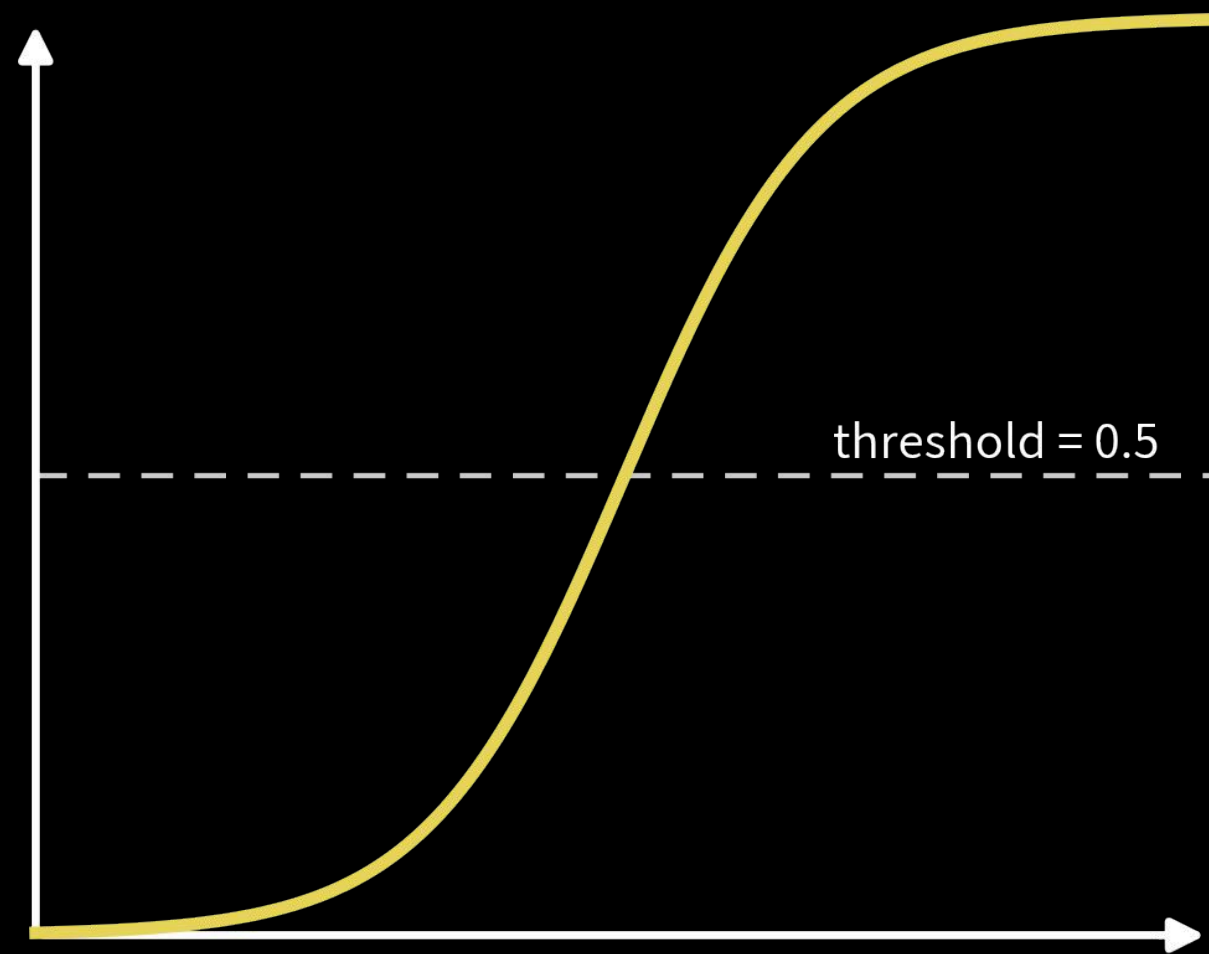
로지스틱회귀, KNN, 의사결정나무, 랜덤 포레스트

주요 알고리즘

알고리즘	핵심 아이디어	장점	단점
로지스틱 회귀	선형 결합 + 시그모이드로 확률 추정	해석 쉬움, 빠름	비선형 관계에 약함
KNN	가까운 k개 이웃의 다수결	직관적, 학습 불필요	예측 느림, 차원의 저주
의사결정나무	특성 기준 분기로 규칙 학습	해석 쉬움, 비선형 가능	과적합 위험
랜덤 포레스트	여러 의사결정 나무의 앙상블	높은 성능, 과적합 방지	해석 어려움, 학습 느림

분류 알고리즘

로지스틱 회귀



선형결합(가중합)을 시그모이드 함수에 통과시켜 0~1 사이 확률을 출력,
임계값(보통 0.5) 기준으로 클래스를 결정

작동 방식

- 입력 특성의 가중합 $z = w_1x_1 + w_2x_2 + \dots + b$ 계산
- 시그모이드 함수로 z 를 확률 $P(y=1|x)$ 로 변환
- 확률이 임계값보다 크면 1, 작으면 0으로 분류
- 경사하강법으로 손실(Log Loss)을 최소화하며 학습

장점

- 계수의 의미가 명확하여 결과 해석이 쉬움
- 학습·예측 속도가 빠름

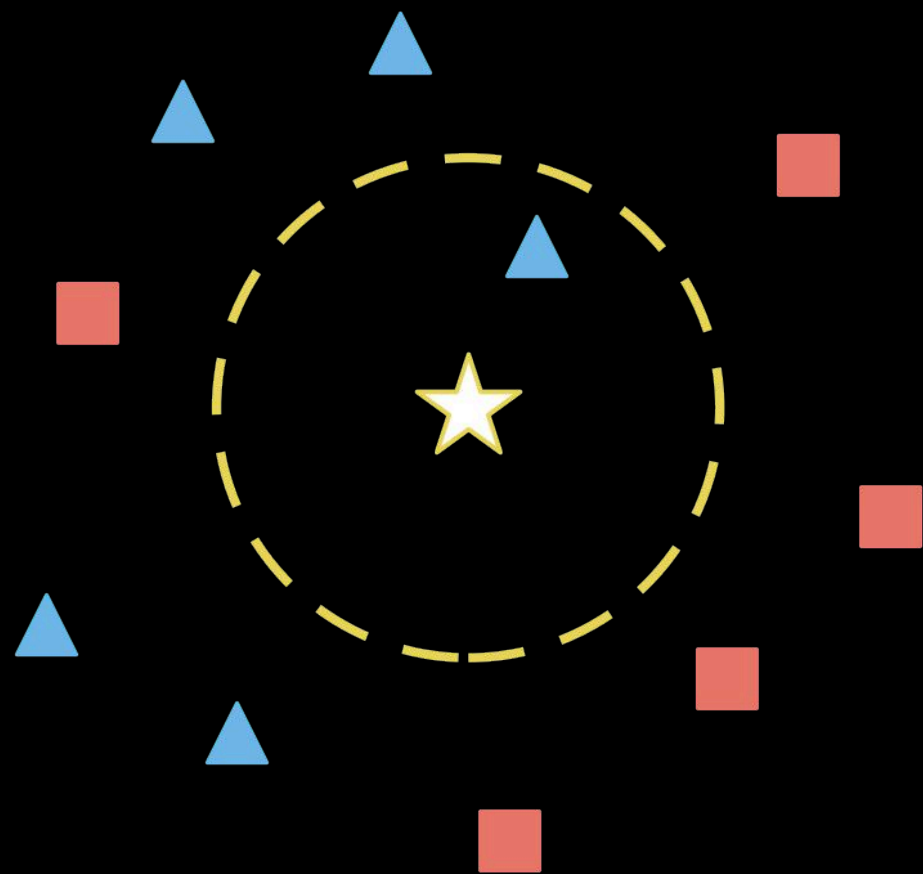
단점

- 특성과 클래스 간 관계가 비선형이면 성능 한계

분류 알고리즘

K-최근접 이웃 (KNN)

▲ 클래스 A ■ 클래스 B ★ 새 데이터



새로운 데이터가 들어오면, 가장 가까운 K개의 이웃 데이터를 찾아
그중 다수를 차지하는 클래스로 예측

작동 방식

- 새 데이터 포인트와 모든 학습 데이터 간의 거리 계산
- 거리가 가까운 순으로 K개의 이웃을 선택
- K개 이웃의 클래스 중 다수결로 예측 클래스 결정
- K값이 작으면 민감(과적합), 크면 둔감(과소적합)

장점

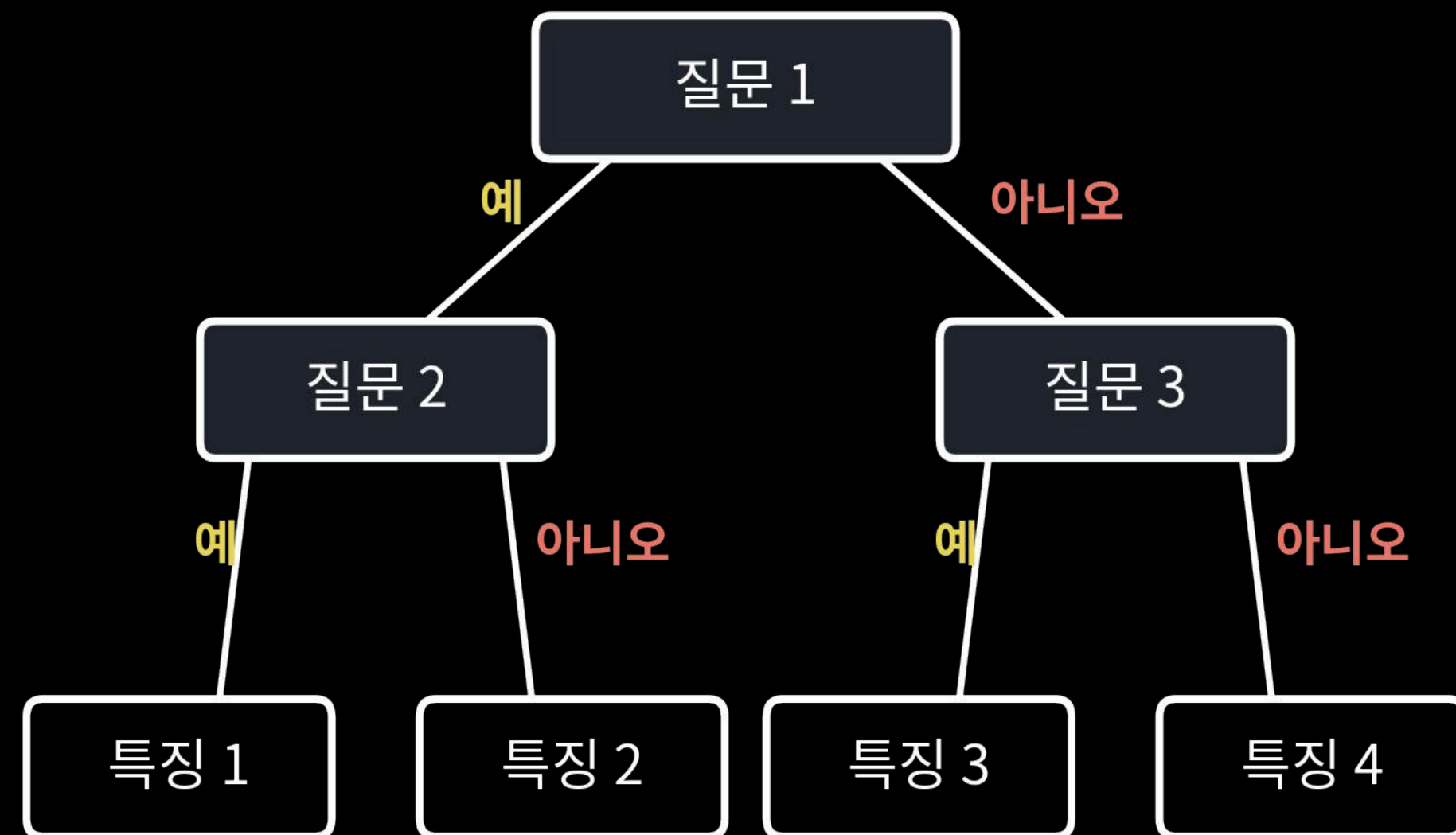
- 별도의 학습 과정이 필요 없음(Lazy learner)
- 비선형 경계 표현 가능

단점

- 데이터·차원이 커질수록 예측 속도 급격히 느려짐

분류 알고리즘

의사결정 나무



특성을 기준으로 데이터를 반복적으로 분기하여,
불순도(Impurity)가 낮아지는 방향으로 나무 형태의 규칙을 학습

작동 방식

- 루트 노드에서 시작해 가장 잘 클래스를 나누는 특성·기준값 탐색
- 지니 계수(Gini) 또는 엔트로피로 분할 품질 평가
- 각 노드를 순도가 높아질 때까지 재귀적으로 분할
- 리프 노드에 도달하면 다수 클래스로 예측

장점

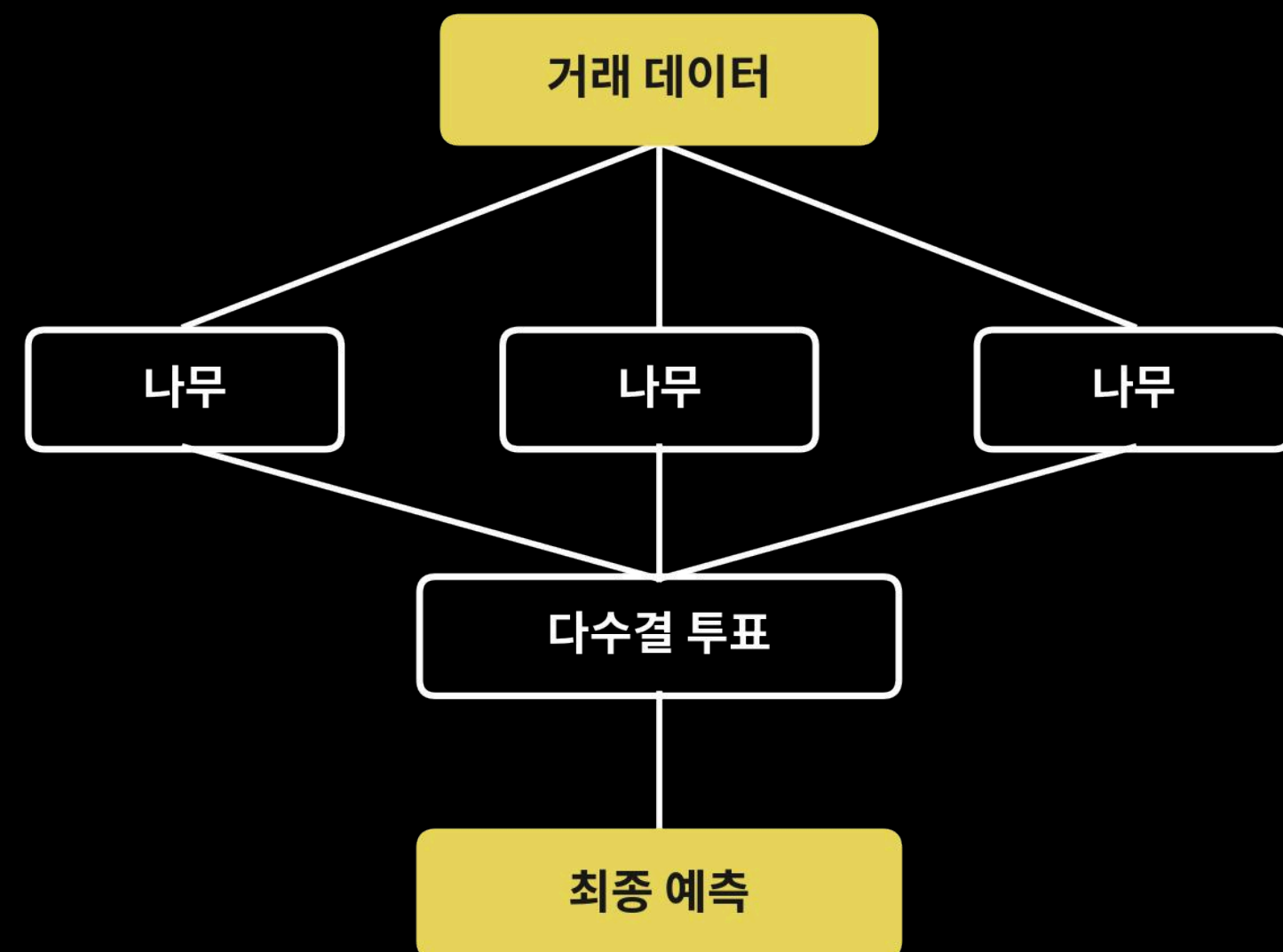
- 결과 해석·시각화가 매우 쉬움

단점

- 깊이가 깊어지면 과적합되기 쉬움
- 데이터 변화에 불안정(분산이 큼)

분류 알고리즘

랜덤 포레스트



여러 개의 의사결정 나무를 무작위로 학습시키고,
다수결이나 평균을 통해 최종 예측을 수행

작동 방식

- 복원 추출으로 여러 개의 학습 데이터 셋 생성
- 각 나무를 만들 때 무작위로 일부 특성(Feature)만 선택하여 분할
- 다수의 의사결정 나무 예측 결과를 모아 다수결로 최종 클래스 결정
- 개별 나무들의 과적합을 줄이고 전체적인 예측 안정성 향상

장점

- 단일 의사결정 나무보다 뛰어난 예측 성능과 높은 일반화 능력

단점

- 모델이 복잡해져 개별 나무들을 해석하기 어려움
- 학습과 예측에 상대적으로 많은 시간 소요

언제 어떤 모델을 쓸까?

결과를 누군가에게 설명해야 한다면?

→ 로지스틱 회귀, 의사결정나무

데이터가 많고 정확도가 최우선이라면?

→ 랜덤 포레스트

데이터가 적고 빠른 프로토타입이 필요하다면?

→ K-최근접 이웃 (KNN)

Chapter 3

모델 평가

혼동 행렬, 정확도, 정밀도, 재현율, F1-score

혼동 행렬

분류 모델의 예측 결과를 실제값과 예측값 기준으로 나눈 표

- TP : 양성을 양성으로 올바르게 예측
- FP : 음성을 양성으로 잘못 예측 (1종 오류)
- FN : 양성을 음성으로 잘못 예측 (2종 오류)
- TN : 음성을 음성으로 올바르게 예측



평가 지표

정확도 (Accuracy)

$$(TP+TN) / \text{전체}$$

: 전체 중 맞게 예측한 비율

정밀도 (Precision)

$$TP / (TP+FP)$$

: 양성 예측 중 실제 양성 비율
→ FP(거짓 경보)를 줄일 때

재현율 (Recall)

$$TP / (TP+FN)$$

: 실제 양성 중 모델이 찾아낸 비율
→ FN(놓친 케이스)을 줄일 때

F1-score

$$2 \times P \times R / (P+R)$$

: 정밀도·재현율의 조화평균

Chapter 4

Python 구현

데이터 전처리, 모델 학습, 평가

Python 구현

데이터 분할 + 스케일링

데이터 로드

```
# 1. 사용할 데이터 불러오기
data = load_breast_cancer() # 예시용 데이터 (유방암 진단)
x, y = data.data, data.target # 특징 데이터, 정답 데이터
```

데이터 분할

```
# 2. Train / Test 분할
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42
) # 학습용 데이터 (Train) / 테스트용 데이터 (Test)
```

스케일링

```
# 3. 데이터 전처리
scaler = StandardScaler() # 표준화
x_train_scaled = scaler.fit_transform(x_train) # 학습 + 변환
x_test_scaled = scaler.transform(x_test) # train을 기준으로 변환
```

Python 구현

모델 정의 + 학습 + 평가

모델 정의

4. 4가지 알고리즘 모델 정의

```
models = {  
    "로지스틱 회귀": LogisticRegression(random_state=42),  
    "KNN": KNeighborsClassifier(),  
    "의사결정 나무": DecisionTreeClassifier(random_state=42),  
    "랜덤 포레스트": RandomForestClassifier(random_state=42),  
}
```

모델 학습 및 평가

5. 모델 학습 및 평가

```
for name, model in models.items():  
    model.fit(x_train_scaled, y_train) # 학습  
    y_pred = model.predict(x_test_scaled) # 예측  
  
score = accuracy_score(y_test, y_pred) # 평가 (정확도)  
print(f"{name} 정확도: {score:.4f}")
```

핵심 함수

함수 / 메서드	역할
<code>train_test_split()</code>	데이터를 학습용 / 평가용으로 분할
<code>LogisticRegression()</code>	로지스틱 회귀 분류 모델 객체 생성
<code>KNeighborsClassifier()</code>	K-최근접 이웃(KNN) 분류 모델 객체 생성
<code>DecisionTreeClassifier()</code>	의사결정 나무 분류 모델 객체 생성
<code>RandomForestClassifier()</code>	랜덤 포레스트(앙상블) 분류 모델 객체 생성
<code>model.fit(x, y)</code>	모든 분류기가 공유하는 모델 학습 인터페이스
<code>model.predict(x)</code>	학습된 모델을 통해 새로운 데이터의 클래스 예측
<code>accuracy_score(y_true, y_pred)</code>	실제 정답과 예측값을 비교하여 모델의 정확도 평가

감사합니다!

과제는 깃허브를 참고해주세요.